

Arrays in Java

14 August 2026 05:55

Java arrays are objects that store multiple elements of the same data type in a single contiguous memory block.

Arrays allow us to group similar data together and access them using an index.

Arrays in Java

Arrays in Java are index based, meaning the first element is at index 0 and the second at index 1 and so on. In Java an array is considered an object of dynamically created class. All arrays in Java:

- Derived from the Object class.
- Array can store primitive values or objects and Java supports both single dimensional and multi-dimensional array.

Single-dimensional array in Java

A single dimensional array in Java is a linear collection of elements of the same data type. It can be declared and instantiated using the following syntax:

```
datatype[] arr; or
datatype []arr; or
datatype arr[];
```

Syntax of array instantiation:

```
Arr = new datatype[size];
```

The **new** keyword allocated the memory dynamically(at runtime) on the heap memory area.

```
package MyArrays;
import java.util.*;
```

```
/**
 * @author (your name)
 * @version (a version number or a date)
 */
public class SDArrayDemo
{
    public static Scanner sc = new Scanner(System.in);
    public static void main(String[] args){
        System.out.print("\n");
        int[] Arr1, i; //all the variable will become an array
```

```

int []Arr2, j; //all the variable will become an array
int Arr3[], k; //all the variables will not become an array
Arr1 = new int[5];
Arr2 = new int[5];
Arr3 = new int[5];

// i = 0; // this type of assignment is not allowed in the array
// j = 0; //j is an array
k = 0;
//Declaring and allocating memory for the array.
int Arr4[] = new int[5];

Arr1[0] = 10;
Arr1[1] = 20;
Arr1[2] = 30;
Arr1[3] = 40;
Arr1[4] = 50;
System.out.print("\nArr1[");
for(k = 0; k < Arr1.length - 1; k++){
    System.out.print(Arr1[k]+", ");
}
System.out.print(Arr1[k]+"]\n");

String []month = {"Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul",
    "Aug", "Sep", "Oct", "Nov", "Dec"};
System.out.print("\nMonths: ");
for(String m : month){ //for each loop is mainly used for Arrays
and Collection
    System.out.print(m+" ");
}
System.out.print("\n");

SDArrayUtility.printSDArray(new int[]{100,200,300,400}, "Anonymous
Array[]: ");

//Don't use in school examination
System.out.print("\nArr1: " + java.util.Arrays.toString(Arr1));

}
}

```

Output:

```
BlueJ: Terminal Window - Kritin11
Options

Arr1[10, 20, 30, 40, 50]

Months: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec

Anonymous Array[]:
[100, 200, 300, 400]

Arr1: [10, 20, 30, 40, 50]
```

ArrayIndexOutOfBoundsException

In Java, the `ArrayIndexOutOfBoundsException` is a runtime exception. Thrown by the Java Virtual Machine (JVM) when attempting to access an invalid index of an array. This exception occurs if the index is negative, Equal to the size of the array or greater than the size of the array while traversing the array.

Homework:

1: Ordered Insertion in an Array (Without Sorting)

Write a program in Java to input N integers into an array in ascending order without using any sorting algorithm.

The user may enter the numbers in any order. As each number is entered, insert it into its correct position in the array by shifting the larger elements one position to the right.

Display the array after every insertion and finally print the complete sorted array.

Sample Input

Enter number of elements: 6

Enter number: 20

Enter number: 10

Enter number: 15

Enter number: 5

Enter number: 18

Enter number: 12

Output

After inserting 20

[20]

After inserting 10

[10, 20]

After inserting 15

[10, 15, 20]

After inserting 5

[5, 10, 15, 20]

After inserting 18

[5, 10, 15, 18, 20]

After inserting 12

[5, 10, 12, 15, 18, 20]

Restrictions

- Do not use Arrays.sort().
- Do not use any Collection classes.
- Insert each element at its proper position while taking input.

2: Second Highest Distinct Element

Write a program to input N integers into an array and determine the second highest distinct element without sorting the array.

If the second highest distinct element does not exist, display an appropriate message.

Sample Input 1

8

15 22 11 45 45 10 39 22

Output

Highest = 45

Second Highest = 39

Sample Input 2

5

20 20 20 20 20

Output

Second Highest Element does not exist.

Restrictions

- Do not sort the array.
- Traverse the array only once after input.
- Do not use an additional array.

3: Insert an Element at a Given Position

Write a program to create an array of N elements.

Input a new element X and a position P (0-based index). Insert the element at the specified position by shifting the remaining elements to the right.

If the array is already full, display "Insertion not possible."

Example

Original Array

[10, 20, 30, 40, 50]

Enter element : 25

Enter position : 2

Output

Array after insertion

[10, 20, 25, 30, 40, 50]

Restrictions

- Do not create another array.
- Perform insertion using shifting only.

4: Maximum Difference

Write a program to input N integers into an array.

Find the largest positive difference between any two elements such that the larger element appears after the smaller element.

If no such pair exists, print 0.

Example

Input

9

7 1 5 3 6 4 8 2 10

Output

Largest Difference = 9

(10 - 1)

Example

Input

5

10 9 8 7 6

Output

Largest Difference = 0

Restrictions

- Do not sort the array.
- Use only one traversal after taking input.

5: Array Rotation Challenge

Write a program to input N integers into an array.

Input an integer K and rotate the array to the right by K positions.

If K is greater than N, rotate by the effective number of positions.

Example

Input

Array

10 20 30 40 50 60

K = 2

Output

50 60 10 20 30 40

Restrictions

- Do not use another array of size N.
- Perform the rotation manually.

6: Longest Consecutive Increasing Sequence

Write a program to input N integers into an array.

Determine the longest consecutive increasing sequence present in the array.

Print

- Starting index
- Ending index
- Length of the sequence
- Elements of the sequence

Example

Input

12

5 8 10 15 3 7 9 11 13 2 4 6

Output

Longest Consecutive Increasing Sequence

7 9 11 13

Starting Index = 5

Ending Index = 8

Length = 4

Example

Input

8

9 8 7 6 5 4 3 2

Output

Longest Consecutive Increasing Sequence

9

Length = 1

Restrictions

- Do not sort the array.
- Do not use any additional array.
- Solve using a single traversal.

7: Smallest Missing Positive Integer

Write a program in Java to input N integers into an array. The array may contain positive numbers, negative numbers, and zero.

Determine the smallest positive integer that is not present in the array.

Sample Input 1

Enter number of elements: 8

3 4 -1 1 7 2 6 8

Output

Smallest Missing Positive Integer = 5

Sample Input 2

Enter number of elements: 5

1 2 3 4 5

Output

Smallest Missing Positive Integer = 6

Restrictions

- Do not sort the array.
- Do not use any Collection classes (ArrayList, HashSet, etc.).
- Do not use library search methods.
- Solve the problem using array traversal and logical reasoning.

8: Leaders in an Array

An element in an array is called a Leader if it is greater than every element to its right. The last element of the array is always considered a leader.

Write a Java program to input N integers into an array and display all the

leader elements in the order in which they appear.

Sample Input

Enter number of elements: 8

16 17 4 3 5 2 8 1

Output

Leader Elements are:

17

8

1

Sample Input

Enter number of elements: 6

30 25 20 15 10 5

Output

Leader Elements are:

30

25

20

15

10

5

Restrictions

- Do not sort the array.
- Do not use another array to store the leaders.
- Display the leaders in their original order of appearance.
- Aim to solve the problem with a single traversal from right to left.

9. Without sorting,

Write a Program to create an array of size N, and find the second highest and second lowest value from it.